

UNIT IV: Clustering

Q1. Give a thorough explanation of Partitioning Clustering.

Partitioning clustering is a type of **unsupervised learning** method where the dataset is divided into a set of **non-overlapping subsets or clusters**, such that each data point belongs to exactly one cluster.

The main idea is to **partition the dataset into 'k' groups (clusters)** based on similarity, typically measured using **distance metrics** like Euclidean distance.

Feature	Description
Type	Flat clustering (non-hierarchical)
Number of clusters (k)	Must be specified before the algorithm starts
Data point membership	Each data point belongs to exactly one cluster
Objective	Minimize intra-cluster distance and maximize inter-cluster distance
Common Algorithms	K-Means, K-Medoids, CLARANS

Working of Partitioning Clustering (Using K-Means as an Example):

1. **Initialization:** Randomly select k data points as initial centroids.
2. **Assignment Step:** Assign each point to the nearest centroid, forming k clusters.
3. **Update Step:** Compute new centroids by taking the mean of all points in each cluster.
4. **Repeat:** Repeat assignment and update steps until convergence (i.e., centroids do not change).

Example: Consider a dataset with customer information based on **age** and **annual income**.

Age	Income
22	25k
25	27k
45	60k
46	65k
30	30k

Goal: Group customers into 2 clusters (k=2) such as:

- Cluster 1: Younger customers with low income
- Cluster 2: Older customers with high income

Using partitioning clustering, the data points are grouped based on proximity in the feature space.

Advantages: Simple and fast for small to medium-sized datasets. Efficient when clusters are spherical and of similar size.

Disadvantages: Requires prior knowledge of k (number of clusters). Sensitive to the initial choice of centroids. Not suitable for clusters with complex shapes or varying sizes.

Q2. Differentiate Hierarchical Clustering and K-Means Clustering

Criteria	Hierarchical Clustering	K-Means Clustering
Type	Hierarchical (tree-based)	Partition-based (flat clustering)
Cluster Structure	Dendrogram representing nested clusters	Divides data into k non-overlapping flat clusters
Input Required	No need to specify number of clusters beforehand	Must predefine number of clusters k

Approach	Agglomerative (bottom-up) or Divisive (top-down)	Iterative refinement (assignment + update steps)
Visualization	Dendrogram (tree structure) can be plotted for visual analysis	No hierarchical structure is formed
Computational Complexity	Higher: $O(n^2)$ or $O(n^3)$, less efficient for large datasets	Lower: $O(nki)$, faster for large datasets
Flexibility in Shape	Can find arbitrarily shaped clusters	Works best with spherical clusters
Scalability	Poor scalability; suitable for small datasets	Scales well with large datasets
Stability	More stable (doesn't require random initialization)	Sensitive to initial centroid positions
Merge/Split Possibility	Once merged or split, decisions cannot be undone	Centroids and cluster memberships are updated iteratively

Q3. Explain DBSCAN Algorithm for Density-Based Clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a popular **density-based clustering algorithm**. It groups together points that are closely packed (high-density regions) and marks points that lie alone in low-density regions as noise or outliers.

Term	Description
Epsilon (ϵ)	Radius defining neighborhood around a point
MinPts	Minimum number of points required to form a dense region
Core Point	Point with at least MinPts points in its ϵ -neighborhood
Border Point	Point that is within ϵ of a core point but has fewer than MinPts neighbors
Noise Point	Point that is not a core point and not reachable from any core point

Algorithm Steps:

1. **Select an unvisited point** PP.
2. **Find all points** within distance ϵ of PP (its ϵ -neighborhood).
3. **If** the ϵ -neighborhood contains at least MinPts points, **form a cluster** and mark PP as a core point.
4. **Expand the cluster** by recursively adding all density-reachable points (neighbors of neighbors).
5. **If** the ϵ -neighborhood has fewer than MinPts points, mark PP as noise (later it may become a border point).
6. **Repeat** until all points have been visited.

Advantages:

- Can find clusters of arbitrary shape and size.
- Automatically detects noise and outliers.
- No need to specify the number of clusters beforehand.

Disadvantages:

- Sensitive to parameters ϵ and MinPts.
- Difficult to estimate ϵ for datasets with varying densities.
- Performance degrades on high-dimensional data.

Example: Consider a set of points scattered on a plane with dense regions and sparse points. DBSCAN identifies dense clusters by grouping points that have many neighbors within ϵ radius and labels sparse points as noise.

Q4. With Reference to Hierarchical Clustering, Explain the Issue of Connectivity Constraints

In **Hierarchical Clustering**, data points are grouped based on their **similarity** or **distance**, forming a dendrogram (tree-like structure) representing nested clusters.

Connectivity Constraints Issue:

- **Connectivity constraints** refer to the requirement that clusters must be formed from **connected points** based on some neighborhood or similarity criteria.
- In hierarchical clustering, the method often **assumes that clusters are formed by connecting nearby points**, but this assumption can cause problems when:
 1. **Clusters are not well separated:**
Points from different true clusters may be connected by a chain of intermediate points, causing **incorrect merging**.
 2. **Chaining effect:**
Especially in **single linkage** hierarchical clustering, distant clusters may get merged due to a series of points that link them together.
 3. **Ignoring global cluster structure:**
Connectivity-based methods make local decisions without considering the overall cluster shape or density, leading to poor clustering results.

Example: Imagine two dense groups of points separated by a few points in between:

- Single linkage may merge these two groups because the intermediate points create a "bridge."
- This violates the intuitive notion that these are separate clusters.

Summary:

- Connectivity constraints can lead to **incorrect cluster merges** due to local proximity without considering cluster compactness.
- This is a known limitation of hierarchical clustering methods, particularly **single linkage**.
- Alternative linkage methods (complete, average) or density-based clustering (e.g., DBSCAN) address this issue better.

Q5. Explain Applications of Clustering

Clustering is an unsupervised learning technique widely used in various fields to discover natural groupings or patterns within data. Below are key applications:

Application Area	Description and Examples
Customer Segmentation	Grouping customers based on buying behavior, demographics, or preferences to tailor marketing strategies.
Image Segmentation	Dividing images into meaningful regions for object detection or recognition (e.g., medical imaging).
Document Clustering	Organizing large text collections into topics or themes, aiding in information retrieval and summarization.
Anomaly Detection	Identifying outliers or fraud in data such as credit card transactions or network security.
Genomics and Bioinformatics	Clustering genes or proteins with similar functions or expressions to understand biological processes.
Market Research	Analyzing survey data to identify groups with similar opinions or behaviors.
Social Network Analysis	Detecting communities or clusters of users based on interaction patterns.
Recommender Systems	Grouping users or items to improve recommendation quality by finding similar preferences.
Earth Sciences	Classifying regions based on soil types, climate, or vegetation for environmental studies.
Health Care	Grouping patients with similar symptoms or diseases for personalized treatment plans.

Q6. What is Clustering? Clarify the Various Applications of Clustering

Clustering is an **unsupervised machine learning technique** that involves grouping a set of data points into clusters such that points in the same cluster are **more similar** to each other than to those in other clusters. It is used to **discover inherent structures or patterns** in data without predefined labels.

- Groups data based on similarity/distance measures.
- Does not require labeled data.
- Helps in summarizing and understanding large datasets.

[Same as Q5]

Q7. Illustrate Bottom-up and Top-down Hierarchical Clustering Approach with an Example in Detail

Hierarchical clustering builds a hierarchy of clusters and can be done in two main ways:

1. Bottom-up Approach (Agglomerative Clustering)

- **Start:** Each data point is treated as an individual cluster.
- **Process:** Iteratively merge the two closest clusters until only one cluster (or desired number of clusters) remains.
- **Output:** A dendrogram representing the nested grouping of clusters.

2. Top-down Approach (Divisive Clustering)

- **Start:** All data points are in a single cluster.
- **Process:** Recursively split clusters into smaller clusters until each data point is alone or the desired cluster count is reached.
- **Output:** A dendrogram showing splits from the root cluster.

Example: Consider 5 data points: A, B, C, D, E

Point	Coordinates (x,y)
A	(1, 2)
B	(2, 2)
C	(5, 5)
D	(6, 5)
E	(8, 8)

Bottom-up (Agglomerative):

- Step 1: Each point is its own cluster: {A}, {B}, {C}, {D}, {E}
- Step 2: Merge closest clusters: {A} and {B} → {A,B}
- Step 3: Merge closest clusters: {C} and {D} → {C,D}
- Step 4: Merge {C,D} and {E} → {C,D,E}
- Step 5: Finally merge {A,B} and {C,D,E} → {A,B,C,D,E}

Top-down (Divisive):

- Step 1: All points in one cluster: {A, B, C, D, E}
- Step 2: Split into two clusters based on distance: {A, B} and {C, D, E}
- Step 3: Further split {C, D, E} into {C, D} and {E} if desired
- Step 4: Continue splitting until singletons or desired clusters

Summary:

Approach	Start Point	Process	Suitable For
Bottom-up	Single data points	Merge clusters	Small to medium datasets
Top-down	One large cluster	Split clusters	When natural splits exist